



UNIVERSIDAD **Blas Pascal**

TRABAJO FINAL

Integrantes:

Baigorria, Constanza

Rosales Mercado, Morena Valentina

Materia:

Técnicas de Compilación

Profesor:

Ameri Lopez Lozano, Francisco

Fecha:

12/06/2026

Índice

1. Introducción	3
2. Análisis del Problema	3
2.1. Subconjunto de C++ Implementado	3
2.2. Objetivos del Compilador	5
3. Diseño de la Solución	5
3.1. Arquitectura General	5
3.2. Tecnologías Utilizadas	6
3.3. Decisiones de Diseño	7
4. Implementación	7
4.1. Análisis Léxico	7
4.2. Análisis Sintáctico	9
4.3. Análisis Semántico	11
4.4. Generación de Código Intermedio	16
4.5. Optimización de Código	19
4.6. Sistema de Reporte de Errores y Warnings	20
5. Ejemplos y Pruebas	21
5.1. Caso Exitoso	22
6. Dificultades Encontradas y Soluciones Aplicadas	22

7. Conclusiones	23
8. Anexos	24

1. Introducción

El presente informe documenta el diseño e implementación de un compilador para un subconjunto del lenguaje C++, desarrollado como parte del trabajo final de la materia Técnicas de Compilación de la carrera Ingeniería Informática de la Universidad Blas Pascal, ciclo 2026.

El compilador fue construido utilizando ANTLR4 como herramienta de generación de analizadores léxicos y sintácticos, y Java como lenguaje de implementación, siguiendo una arquitectura por fases claramente separadas: análisis léxico, análisis sintáctico, análisis semántico, generación de código intermedio y optimización.

El propósito principal del sistema es transformar programas escritos en un subconjunto controlado de C++ en una representación interna validada, detectando errores léxicos, sintácticos y semánticos, para posteriormente generar código intermedio de tres direcciones (3AC) y aplicar técnicas de optimización que reduzcan la cantidad de instrucciones generadas sin alterar el comportamiento lógico del programa.

2. Análisis del Problema

2.1. Subconjunto de C++ Implementado

El compilador da soporte al siguiente subconjunto del lenguaje C++, definido en la gramática formal **MiLenguaje.g4**:

Categoría	Elementos Soportados

Tipos de Datos	int, double, char, bool, string, void
Estructuras de Control	if, if-else, while, for, break, continue
Declaraciones	Variables simples, arrays, funciones con parámetros
Expresiones	Aritméticas, relaciones, lógicas, incremento/decremento
Otros	Llamadas a funciones, retorno de valores, asignaciones, asignación a arrays, comentarios

2.2. Objetivos del Compilador

- Reconocer y reportar errores léxicos mediante un listener personalizado sobre el lexer generado por ANTLR4.
- Validar la estructura sintáctica del programa y construir el Árbol de Sintaxis Abstracta (AST) correspondiente.
- Garantizar la coherencia semántica: comprobación de tipos, gestión de ámbitos anidados y declaración de símbolos.
- Generar código intermedio basado en instrucciones de tres direcciones (3AC).
- Aplicar técnicas de optimización que reduzcan la cantidad de instrucciones sin modificar la lógica del programa.
- Proveer un sistema de reporte de errores y warnings con diferenciación visual por colores.

3. Diseño de la Solución

3.1. Arquitectura General

La arquitectura del sistema abarca las tres primeras fases críticas de un compilador moderno:

1. Análisis Léxico: Transformación del flujo de caracteres en tokens significativos (palabras clave, identificadores, operadores).

2. **Análisis Sintáctico:** Construcción de un árbol de sintaxis abstracta (AST) que representa la estructura jerárquica del código según las reglas de la gramática.
3. **Análisis Semántico:** Validación de la lógica del programa, incluyendo la comprobación de tipos, la gestión de ámbitos (scopes) y la declaración de variables.
4. **Generador de Código Intermedio:** Traduce el AST ya validado a una representación abstracta lineal e independiente de la máquina, comúnmente estructurada como Código de Tres Direcciones (TAC).
5. **Optimizador de Código:** Analiza y transforma el código intermedio (TAC) generado con el objetivo de mejorar su eficiencia de ejecución y reducir el consumo de recursos (memoria y procesamiento).

Cada módulo opera sobre la salida producida por el anterior, manteniendo una separación clara de responsabilidades.

3.2. Tecnologías Utilizadas

El desarrollo se apoya en herramientas de estándar industrial para garantizar robustez y escalabilidad:

- ANTLR4: Utilizado para la definición de la gramática (MiLenguaje.g4) y la generación automática del lexer y el parser.

- Java y Maven: Lenguaje de implementación y gestor de dependencias que facilita la organización del código en paquetes especializados (com.compilador.semantico).
- Patrón Visitor: Mecanismo empleado para recorrer el árbol de parseo de forma eficiente, permitiendo una separación clara entre la estructura de los datos y la lógica de validación semántica.

3.3. Decisiones de Diseño

Se decidió utilizar ANTLR4 debido a su capacidad para generar automáticamente analizadores léxicos y sintácticos a partir de una gramática formal.

Para el análisis semántico se implementó el patrón Visitor, permitiendo recorrer el árbol de análisis de forma desacoplada de la gramática.

La gestión de símbolos se centralizó en una tabla de símbolos jerárquica que permite administrar ámbitos anidados.

4. Implementación

4.1. Análisis Léxico

El analizador léxico fue generado automáticamente por ANTLR4 a partir de las reglas del lexer definidas en MiLenguaje.g4. La clase resultante, MiLenguajeLexer, transforma el flujo de caracteres de entrada en una secuencia de tokens clasificados.

Tokens Reconocidos por el Lexer

Categoría	Tokens
Palabras Reservadas	int, double, char, bool, string, void, if, else, while, for, break, continue, return, true, false
Operadores Aritméticos	+ - * / %
Operadores Relacionales	== != < > <= >=
Operadores Lógicos	&& !
Operadores Incrementales	++ --
Delimitadores	() [] {} ; ,
Literales Numéricos	NUM (int), DECIMAL (float)
Literales Textuales	CHARACTER ('x'), CADENA ("texto")

Identificadores	ID: letra o _ seguido de letras, dígitos o _
Comentarios	// /* */
Errores Léxicos	OTRO: cualquier caracter no reconocido, reportado vía ErrorListener

4.2. Análisis Sintáctico

La gramática fue implementada mediante ANTLR4 a partir de las reglas del parser en MiLenguaje.g4. La clase resultante, MiLenguajeParser, verifica que la secuencia de tokens cumpla la gramática definida y construye un Árbol de Sintaxis Abstracta (AST).

Reglas Principales de la Gramática

- programa: punto de entrada; secuencia de declaraciones seguida de EOF.
- declaracion: declaracion de variable o declaracion de funcion.
- declaracionVariable: tipo ID (= expresion)? ; | tipo ID[NUM]; (arrays)
- declaracionFuncion: tipo ID (parametros?) bloque
- bloque: { sentencia* }: genera nuevo scope en el análisis

semántico.

- sentencia: declaración, asignación, incremento, llamada a función, if, while, for, break, continue, return.
- expresion: jerarquía completa de precedencia mediante alternativas etiquetadas (#ExprOr, #ExprAnd, etc.).

Al igual que en el análisis léxico, se registra un BaseErrorListener personalizado sobre el parser que acumula los errores sintácticos. Si se detectan errores, el proceso de compilación se interrumpe antes de continuar con el análisis semántico.

La visualización del AST se implementa mediante la clase TreeViewer de ANTLR4, que abre una ventana gráfica (Swing) mostrando el árbol sintáctico de forma interactiva y navegable, con escala ajustable.

4.3. Análisis Semántico

La validación semántica es el mecanismo que garantiza la integridad de los tipos y la coherencia de los ámbitos (scopes). El componente central es el SemanticAnalyzer, que extiende MiLenguajeBaseVisitor e implementa el patrón Visitor para recorrer el AST en profundidad.

Arquitectura de la Tabla de Símbolos

La tabla de símbolos administra cuatro categorías de información:

- Variables.
- Funciones.

- Tipos.
- Ámbitos.

Cada bloque de código genera un nuevo scope enlazado al scope padre, permitiendo la resolución correcta de nombres en contextos anidados. Al salir del bloque, el scope se destruye y sus símbolos locales quedan fuera de alcance.

Implementación del Patrón Visitor

El SemanticAnalyzer opera como el cerebro del compilador. Utiliza el patrón Visitor para realizar un recorrido en profundidad sobre el AST. Su función principal es validar que el programa tenga "sentido lógico", transformando un árbol puramente sintáctico en una estructura de ejecución válida.

Los nodos de sentencias producen efectos sobre la tabla de símbolos mientras que los nodos de expresiones retornan el tipo resultante de la evaluación.

El SemanticAnalyzer sigue una convención de retorno dual para distinguir los dos tipos de nodos del AST:

- Nodos de Sentencia (visitDeclaracion, visitSentencialf):
Devuelve null. Su propósito es generar efectos secundarios en la SymbolTable o realizar verificaciones de control de flujo.
- Nodos de Expresión (visitExprAditiva, visitExprIdentificador):
Devuelven un String que representa el tipo resultante de la

expresión.

Cuando el motor encuentra una declaracion de variable, el flujo es:
visit(expresion) -> cadena recursiva de visitas -> tipo resultante sube
por la pila -> TypeSystem verifica compatibilidad -> si válido, se crea
Symbol y se registra en SymbolTable.

Sistema de Tipos y Reglas de Compatibilidad

Se implementó un sistema de compatibilidad basado en promociones
numéricas.

Tipo Destino	Tipos Aceptados (Origen)	Nota
int	int	Identidad estricta.
double	int, float, double	Máxima precisión numérica; acepta ampliación.
float	int, float, double	Permite ampliación y acepta precisión superior según contexto.
char	char	Tipo incompatible

		con numéricos.
bool	bool	Requerido para condiciones en if, while y for.
string	string	Solo compatible consigo mismo.

Validaciones Implementadas

Variables

- Redeclaración en el mismo ámbito: Se detecta cuando una variable ya registrada en el scope actual intenta declararse nuevamente.
- Uso sin declarar: Se reporta error cuando se referencia un identificador que no existe en ningún scope de la cadena jerárquica.
- Uso sin inicialización: Se emite un warning cuando una variable fue declarada pero no tiene valor asignado al momento de ser utilizada.
- Variables declaradas pero nunca usadas: Se emite un warning al finalizar el análisis del scope si alguna variable fue declarada y nunca referenciada.

Tipos

- Incompatibilidad en asignaciones: Se verifica que el tipo de la expresión del lado derecho sea compatible con el tipo declarado de la variable destino según las reglas del TypeSystem.
- Operandos inválidos para operadores específicos: Se controla que los operandos de cada operador sean de tipos coherentes (por ejemplo, que no se sumen un char y un double).
- Condiciones que no evalúan a bool: Las expresiones usadas como condición en if, while y for deben resolverse al tipo bool; de lo contrario se reporta error.

Funciones

- Declaración duplicada: Se detecta cuando una función ya definida en el scope global intenta declararse nuevamente con el mismo nombre.
- Cantidad incorrecta de argumentos: Se verifica que el número de argumentos en la llamada coincida con el número de parámetros declarados en la firma.
- Tipos incompatibles en argumentos: Se controla que cada argumento pasado sea compatible con el tipo del parámetro correspondiente.
- return fuera de contexto de función: Se reporta error si se encuentra una sentencia return fuera del cuerpo de una función.

- Incompatibilidad entre tipo de retorno y tipo declarado: Se verifica que la expresión retornada sea compatible con el tipo de retorno declarado en la firma de la función.
- Funciones void que retornan valor: Se reporta error si una función declarada como void contiene un return con expresión.
- Funciones no-void sin return: Se reporta error si una función con tipo de retorno distinto de void no tiene ninguna sentencia return alcanzable.

Control de Flujo

- break y continue fuera de bucle: Se reporta error si estas sentencias aparecen fuera de un contexto for, while o for.
- Validación de alcanzabilidad post-return: se detecta código que no puede ejecutarse porque aparece después de una sentencia return dentro de un bloque.

4.4. Generación de Código Intermedio

Utilizando el componente `CodigoTresDir`, el sistema recorre los nodos validados y genera Código de Tres Direcciones (3AC), donde cada instrucción opera con un máximo de tres operadores (dos fuentes y un destino).

4.5. Optimización de Código

La fase de optimización analiza el código 3AC generado e implementa

cuatro técnicas de reducción de instrucciones. El objetivo es mejorar el rendimiento sin alterar el comportamiento lógico del programa:

1. Propagación de constantes: Identifica valores fijos para reducir el uso de temporales innecesarios.
2. Simplificación de expresiones: Realiza cálculos en tiempo de compilación.
3. Eliminación de código muerto: Remueve bloques inalcanzables.
4. Eliminación de temporales muertos: Suprime asignaciones circulares o inútiles.

El impacto de estas técnicas se refleja en las métricas de rendimiento del sistema:

Métrica	Valor
Instrucciones Originales	51
Instrucciones Optimizadas	42
Instrucciones Eliminadas	9
Reducción de Código	17.65%

Optimización por Pasadas Múltiples hasta Punto Fijo

El optimizador no aplica las técnicas una única vez, sino que itera sobre el código intermedio en múltiples pasadas sucesivas hasta que una pasada completa no produce ningún cambio en el conjunto de instrucciones.

El sistema genera automáticamente un archivo de salida por cada pasada ejecutada. Así, para el archivo de entrada `ejemplo_correcto.cpp`, se producen los archivos `ejemplo_correcto_optimizacion_pasada_1.txt`, `ejemplo_correcto_optimizacion_pasada_2.txt`, y así sucesivamente hasta que el proceso converge.

En el caso del programa de prueba `ejemplo_correcto.cpp`, el proceso converge en 2 pasadas, pasando de 51 instrucciones originales a 42 instrucciones optimizadas, con una reducción del 17,65%.

4.6. Sistema de Reporte de Errores y Warnings

El compilador implementa un sistema de reporte de errores con diferenciación visual por colores en la salida de consola, cumpliendo el requerimiento especificado en el enunciado del trabajo:

Nivel	Color en Consola	Descripción
ÉXITO	Verde	La fase de compilación se completó sin problemas.
WARNING	Amarillo	Situaciones no críticas: variable no usada, uso sin inicializar, etc.
ERROR	Rojo	Errores críticos que impiden la compilación correcta del programa.

La estrategia de acumulación garantiza que el desarrollador reciba un reporte completo de todos los problemas del programa en una sola ejecución del compilador, evitando el ciclo de corregir-recompilar-descubrir-siguiente-error.

5. Ejemplos y Pruebas

Para validar el funcionamiento del compilador se utilizaron dos archivos de prueba que cubren los dos escenarios posibles: compilación exitosa y detección de errores múltiples.

5.1. Caso Exitoso

El siguiente programa fue utilizado como caso de prueba sin errores. Cubre variables globales, arrays, operaciones aritméticas, llamadas a función y estructuras de control:

Salida de ejemplo_correcto.cpp

```
=== 1. ANALISIS LEXICO ===
Análisis lexico completado sin errores.
Tokens procesados: 167

=== 2. ANALISIS SINTACTICO ===
Análisis sintactico completado sin errores.
Arbol sintactico generado correctamente

=== 3. VISUALIZACION DEL AST ===
Ventana del arbol sintactico abierta

=== 4. ANALISIS SEMANTICO ===
Tabla de simbolos construida:

=== TABLA DE SIMBOLOS ===
NOMBRE      TIPO      CATEGORIA  LINEA  COLUMNA  AMBITO      DETALLES
-----
contadorGlobal  int      variable   5      0      global     [private]
valorPi        double   variable   6      0      global     [private]
inicial        char     variable   7      0      global     [private]
activo         bool     variable   8      0      global     [private]
sumar          int      funcion    11     0      global     [private] [int, int]
main           int      funcion    19     0      global     [private] []
a              int      parametro  11     14     sumar
b              int      parametro  11     21     sumar
resultado      int      variable   12     4      sumar     [private]
estado         int      variable   20     4      main      [private]
temp           int      variable   21     4      main      [private]
numeros        int      variable   22     4      main      [arr:3] [private]
auxiliar       int      variable   50     8      main      [private]

Análisis semantico completado sin errores.
```

```
=== 5. GENERACION DE CODIGO INTERMEDIO ===  
Iniciando recorrido del AST conCodigoTresDir...
```

```
0: PROGRAMA_INICIO:  
1: // Declaracion de variables globales  
2: DECLARE contadorGlobal int  
3: DECLARE valorPi double  
4: DECLARE inicial char  
5: DECLARE activo bool  
6: func_sumar:  
7: PARAM a int  
8: PARAM b int  
9: DECLARE resultado int  
10: t1 = a + b  
11: resultado = t1  
12: t2 = contadorGlobal + 1  
13: contadorGlobal = t2  
14: return resultado  
15: func_main:  
16: DECLARE estado int  
17: DECLARE temp int  
18: DECLARE numeros[3] int  
19: contadorGlobal = 0  
20: valorPi = 3.14  
21: inicial = 'M'  
22: numeros[0] = 10  
23: numeros[1] = 20  
24: numeros[2] = 30  
25: t3 = numeros[0]  
26: t4 = numeros[1]  
27: t5 = t3 + t4  
28: temp = t5  
29: t6 = temp * 2  
30: temp = t6  
31: t7 = temp / 3  
32: temp = t7  
33: t8 = temp % 5
```

```

34: temp = t8
35: CALL func_sumar, temp, 5
36: estado = RETURN_VALUE
37: contadorGlobal = estado
38: valorPi = temp
39: inicial = 'X'
40: t9 = estado > 0
41: if t9 goto THEN_1
42: goto END_IF_1
43: THEN_1:
44: DECLARE auxiliar int
45: t10 = estado + 10
46: auxiliar = t10
47: estado = auxiliar
48: END_IF_1:
49: return estado
50: PROGRAMA_FIN:
Codigo intermedio guardado en: ejemplo_correcto_codigo_intermedio.txt

```

=== 6. OPTIMIZACION DE CODIGO ===

Aplicando optimizaciones al codigo intermedio...

Pasada 1 guardada en: ejemplo_correcto_optimizacion_pasada_1.txt

Pasada 2 guardada en: ejemplo_correcto_optimizacion_pasada_2.txt

Optimizacion completada:

```

Instrucciones originales: 51
Instrucciones optimizadas: 42
Instrucciones eliminadas: 9
Reduccion de codigo:      17,65%
Pasadas ejecutadas:      2

```

Detalle por pasada:

	+	-----	+	-----	+	-----	+	-----	+
		Pasada		Antes		Despues		Eliminadas	
	+	-----	+	-----	+	-----	+	-----	+
		Pasada 1		51		42		9	
		Pasada 2		42		42		0	(menor)
	+	-----	+	-----	+	-----	+	-----	+

Codigo optimizado:

```
0: PROGRAMA_INICIO:
1: // Declaracion de variables globales
2: DECLARE contadorGlobal int
3: DECLARE valorPi double
4: DECLARE inicial char
5: DECLARE activo bool
6: func_sumar:
7: PARAM a int
8: PARAM b int
9: t1 = a + b
10: resultado = t1
11: t2 = contadorGlobal + 1
12: contadorGlobal = t2
13: return resultado
14: func_main:
15: contadorGlobal = 0
16: valorPi = 3.14
17: inicial = 'M'
18: numeros[0] = 10
19: numeros[1] = 20
20: numeros[2] = 30
21: t3 = numeros[0]
22: t4 = numeros[1]
23: t5 = t3 + t4
24: t6 = t5 * 2
25: t7 = t6 / 3
26: t8 = t7 % 5
27: temp = t8
28: CALL func_sumar, temp, 5
29: estado = RETURN_VALUE
30: contadorGlobal = estado
31: valorPi = temp
32: inicial = 'X'
33: t9 = estado > 0
```

```
34: if t9 goto THEN_1
35: goto END_IF_1
36: THEN_1:
37: t10 = estado + 10
38: estado = t10
39: END_IF_1:
40: return estado
41: PROGRAMA_FIN:
Codigo optimizado guardado en: ejemplo_correcto_codigo_optimizado.txt
```

=== 7. RESUMEN DE COMPILACION ===

```
Archivo procesado:      ejemplo_correcto.cpp
Tokens analizados:      167
Simbolos en tabla:      13
Instrucciones generadas: 51
Instrucciones optimizadas: 42
Archivo codigo intermedio: ejemplo_correcto_codigo_intermedio.txt
Archivo codigo optimizado: ejemplo_correcto_codigo_optimizado.txt
COMPILACION Y OPTIMIZACION EXITOSA!
```

Salida de ejemplo_correcto_codigo_optimizado.txt

```
// Codigo optimizado generado
0: PROGRAMA_INICIO:
1: // Declaracion de variables globales
2: DECLARE contadorGlobal int
3: DECLARE valorPi double
4: DECLARE inicial char
5: DECLARE activo bool
6: func_sumar:
7: PARAM a int
8: PARAM b int
9: t1 = a + b
10: resultado = t1
11: t2 = contadorGlobal + 1
12: contadorGlobal = t2
13: return resultado
14: func_main:
15: contadorGlobal = 0
16: valorPi = 3.14
17: inicial = 'M'
18: numeros[0] = 10
19: numeros[1] = 20
20: numeros[2] = 30
21: t3 = numeros[0]
22: t4 = numeros[1]
23: t5 = t3 + t4
24: t6 = t5 * 2
25: t7 = t6 / 3
```

```
26: t8 = t7 % 5
27: temp = t8
28: CALL func_sumar, temp, 5
29: estado = RETURN_VALUE
30: contadorGlobal = estado
31: valorPi = temp
32: inicial = 'X'
33: t9 = estado > 0
34: if t9 goto THEN_1
35: goto END_IF_1
36: THEN_1:
37: t10 = estado + 10
38: estado = t10
39: END_IF_1:
40: return estado
41: PROGRAMA_FIN:
```

5.2. Caso con Errores

Este archivo fue diseñado para verificar que el sistema de reporte detecta y acumula correctamente múltiples errores semánticos y warnings en una sola ejecución, sin interrumpirse al encontrar el primero.

```
=== 1. ANALISIS LEXICO ===
Análisis lexico completado sin errores.
Tokens procesados: 132

=== 2. ANALISIS SINTACTICO ===
Análisis sintactico completado sin errores.
Árbol sintactico generado correctamente

=== 3. VISUALIZACION DEL AST ===
Ventana del árbol sintactico abierta

=== 4. ANALISIS SEMANTICO ===
Tabla de símbolos construida:

=== TABLA DE SIMBOLOS ===
NOMBRE          TIPO      CATEGORIA  LINEA  COLUMNA  AMBITO      DETALLES
-----
variableGlobal  int       variable   2       0        global     [private]
valorGlobal     double    variable   4       0        global     [private]
caracterGlobal  char      variable   5       0        global     [private]
miFuncion       int       funcion    8       0        global     [private] [int, double]
funcionVoid     void      funcion    32      0        global     [private] []
main            int       funcion    45      0        global     [private] []
parametro1      int       parametro  8       18       miFuncion
parametro2      double    parametro  8       37       miFuncion
variableLocal   int       variable   9       4        miFuncion  [private]
variableNoUsada1int  variable  13       4        miFuncion  [private]
variableNoUsada2string variable  14       4        miFuncion  [private]
variableNoUsada3double variable  15       4        miFuncion  [private]
x               int       variable   33      4        funcionVoid [private]
y               int       variable   34      4        funcionVoid [private]
z               int       variable   35      4        funcionVoid [private]
resultado       int       variable   46      4        main       [private]
valor           int       variable   47      4        main       [private]

ERRORES SEMANTICOS:
Error: La variable 'variableGlobal' ya esta declarada en el ambito 'global' (línea 3, columna 0)
Error: La variable 'variableLocal' ya esta declarada en el ambito 'miFuncion' (línea 10, columna 4)
Error: Variable 'variableFantasma' no declarada en el ambito 'miFuncion' (línea 21)
Error: No se puede asignar valor a 'miFuncion' porque no es una variable (línea 24)
Error: Variable 'w' no declarada en el ambito 'funcionVoid' (línea 41)
Error: Variable 'variableFinal' no declarada en el ambito 'main' (línea 53)
WARNINGS SEMANTICOS:
Warning: Variable 'variableNoUsada1' declarada pero nunca utilizada en el ambito 'miFuncion' (línea 13)
Warning: Variable 'variableNoUsada2' declarada pero nunca utilizada en el ambito 'miFuncion' (línea 14)
Warning: Variable 'variableNoUsada3' declarada pero nunca utilizada en el ambito 'miFuncion' (línea 15)
Warning: Variable 'z' declarada pero nunca utilizada en el ambito 'funcionVoid' (línea 35)
El código tiene warnings, pero se puede continuar.
Compilacion detenida debido a errores semanticos.
```

6. Dificultades Encontradas y Soluciones Aplicadas

6.1. Generación de Código Intermedio: Manejo de Expresiones con Arrays

Al generar código de tres direcciones para expresiones que involucran accesos a arrays (por ejemplo, `numeros[0] + numeros[1]`), el generador

producía instrucciones temporales separadas para cada acceso antes de poder combinarlos. Esto generaba temporales adicionales que no siempre eran necesarios, aumentando la cantidad de instrucciones del código intermedio y dejando trabajo extra para la fase de optimización.

Como solución, se estableció una convención en el visitor `CodigoTresDir` donde cada subexpresión retorna el nombre del temporal o valor que la representa. De esta forma, el acceso a un array siempre produce un temporal explícito que puede ser referenciado directamente en instrucciones posteriores. Si bien esto incrementa levemente el número de instrucciones generadas, garantiza que el código intermedio sea correcto y que el optimizador pueda luego eliminar los temporales muertos resultantes.

6.2. Optimización: Condición de Parada Incorrecta en el Bucle de Pasadas

La implementación inicial del bucle de optimización utilizaba el tamaño de la lista de instrucciones como única condición de convergencia. Esta condición provocaba que el bucle se detuviera prematuramente. El caso problemático ocurría cuando la propagación de constantes modifica el contenido de instrucciones (por ejemplo, reemplazando $t1 = 5 + 3$ por $t1 = 8$) sin cambiar la cantidad de líneas. En la siguiente pasada, esa constante 8 podría haber habilitado nuevas eliminaciones de temporales muertos, pero el bucle ya había terminado, dejando el código sin optimizar completamente.

Por ello, se reemplazó la comparación por tamaño con una comparación por contenido completo, serializando la lista de instrucciones a un String antes y después de cada pasada. Con esta corrección, el optimizador continúa iterando mientras cualquier instrucción haya cambiado (en contenido o en cantidad), garantizando que se alcance el verdadero punto fijo. En el programa de prueba ejemplo_correcto.cpp, esto resultó en 2 pasadas completas: la primera eliminó 9 instrucciones y la segunda confirmó que no quedaban más optimizaciones posibles.

6.3. Optimización: Condición de Parada Incorrecta en el Bucle de Pasadas

Durante la propagación de constantes, el optimizador sustituye correctamente los temporales en expresiones aritméticas, pero aplicaba esa misma sustitución dentro de los argumentos de instrucciones CALL. Esto producía que variables con nombre significativo (como temp) fueran reemplazadas por temporales internos (como t8) en la llamada a función. El temporal t8 es un registro de vida corta que ya no tiene validez semántica en ese punto del programa. Pasar t8 como argumento en lugar de temp es semánticamente incorrecto, ya que temp puede tener otros usos posteriores que dependen de ese valor.

Como solución, se modificó el método `sustituirArgsCall()` para que, al resolver argumentos de una llamada, solo propague la sustitución si el resultado es un literal (número o constante), nunca otro temporal. Esta

corrección preserva la semántica del programa y asegura que los argumentos de las llamadas a función siempre referencian variables con nombre válido.

6.4. Testing: Ajuste del Test de Convergencia para Programa Óptimo

Al diseñar el test `testProgramaYaOptimoUnaPasada`, se asumió que un programa sin expresiones constantes ni código muerto terminaría en exactamente 1 pasada. Sin embargo, el test fallaba. El motivo fue que el test construía la variable `x` como local de `func_main` mediante el helper `envolverEnMain()`. El optimizador, al recorrer el código en sentido inverso (backward scan), detectaba que el `DECLARE x` aparecía antes de que `x` fuera "necesaria" desde el retorno, y lo eliminaba en la pasada 1. Esto provocaba un cambio de contenido que disparaba una segunda pasada de verificación, comportamiento completamente correcto del algoritmo.

Se rediseñó el test para usar una variable declarada a nivel global (fuera de cualquier función), dado que el optimizador conserva siempre los `DECLARE` globales. De este modo, el programa verdaderamente no tiene nada que modificar desde la primera pasada y el test verifica correctamente la convergencia en 1 pasada.

7. Conclusiones

El desarrollo de este compilador permitió recorrer en su totalidad las fases

que componen un compilador moderno, integrando teoría de lenguajes formales, diseño de algoritmos y pruebas de software en un único sistema funcional.

El sistema detecta y acumula errores léxicos, sintácticos y semánticos sin interrumpirse ante el primero, genera código de tres direcciones válido y lo optimiza mediante cuatro técnicas que, sobre el programa de prueba, redujeron las instrucciones de 51 a 42 (17,65%) en 2 pasadas hasta alcanzar el punto fijo.

La dificultad más significativa fue descubrir que una condición de parada basada solo en el tamaño de la lista de instrucciones no es suficiente: el optimizador puede modificar el contenido de una instrucción sin eliminarla, habilitando nuevas optimizaciones en la pasada siguiente. Corregir esto comparando el contenido completo entre pasadas fue el cambio conceptual más importante del proyecto.

El sistema construido constituye una base funcional y extensible que demuestra la viabilidad de implementar un compilador completo aplicando las herramientas y metodologías estudiadas durante la materia.

8. Anexos

Estructura del Proyecto

demo/

|

|— pom.xml

```
├── compilador.sh
|
├── ejemplo_correcto.cpp
├── ejemplo_multiples_erroses.cpp
|
├── ejemplo_correcto_codigo_intermedio.txt
├── ejemplo_correcto_codigo_optimizado.txt
├── ejemplo_correcto_optimizacion_pasada_1.txt
├── ejemplo_correcto_optimizacion_pasada_2.txt
|
└── src/
    ├── main/
    |   ├── antlr4/com/compilador/
    |   |   └── MiLenguaje.g4
    |   |
    |   └── java/com/compilador/
    |       |
    |       ├── App.java
    |       |
    |       ├── MiLenguajeLexer.java
    |       ├── MiLenguajeParser.java
    |       ├── MiLenguajeVisitor.java
    |       └── MiLenguajeBaseVisitor.java
```

```
|   |
|   |— semantico/
|   |   |— SemanticAnalyzer.java
|   |   |— SymbolTable.java
|   |   |— Scope.java
|   |   |— Symbol.java
|   |   |— TypeSystem.java
|   |   |— SemanticError.java
|   |
|   |— codigoIntermedio/
|   |   |—CodigoTresDir.java
|   |   |— Instruccion.java
|   |
|   |— optimizacion/
|       |— Optimizador.java
|
|— test/
    |— java/com/compilador/
        |— AppTest.java
        |— OptimizadorTest.java
```

Repositorio GitHub

<https://github.com/MValentinaMercado/TC-ProyectoFinal>